

Variabilité en jeu vidéo

Dans le développement de jeux vidéo, l'utilisation de nombres aléatoires est essentielle pour introduire de la variabilité et de l'imprévisibilité dans le gameplay. On peut utiliser des nombres aléatoires pour :

- Générer des positions aléatoires pour les objets ou ennemis.
- Déterminer des comportements ou actions aléatoires des personnages non-joueurs (PNJ).
- Créer des événements aléatoires qui affectent le joueur.
- Varier les statistiques des personnages ou objets (comme les points de vie, la puissance d'attaque, etc.).
- Ajouter de la diversité dans les niveaux ou environnements de jeu.
- Simuler des éléments de hasard dans les mécaniques de jeu, comme les chances de réussite ou d'échec.
- Générer des récompenses aléatoires pour les joueurs.

1.1 Génération de nombres aléatoires avec Unity

Pour générer un nombre aléatoire avec Unity, vous pouvez utiliser la classe `Random` ou la méthode statique `Random.Range()`. Voici quelques exemples pour vous aider à comprendre comment générer des nombres aléatoires dans différents contextes.

1.1.1 Générer un nombre entier aléatoire

Pour générer un nombre entier aléatoire entre deux valeurs (inclusif pour la valeur minimale et exclusif pour la valeur maximale), vous pouvez utiliser la méthode `Random.Range(int min, int max)`. En spécifiant le type de la variable comme `int`, vous obtiendrez un entier aléatoire.

```
int randomInt = Random.Range(0, 10); // Génère un entier aléatoire entre 0 et 9
Debug.Log("Nombre entier aléatoire : " + randomInt);
```

1.1.2 Générer un nombre décimal aléatoire

Pour générer un nombre décimal (float) aléatoire entre deux valeurs, vous pouvez utiliser la méthode `Random.Range(float min, float max)`. En spécifiant le type de la variable comme `float`, vous obtiendrez un nombre décimal aléatoire.

```
float randomFloat = Random.Range(0.0f, 10.0f); // Génère un nombre décimal aléatoire entre 0.0 et 10.0
Debug.Log("Nombre décimal aléatoire : " + randomFloat);
```

1.1.3 Variété dans les positions

Vous pouvez utiliser des nombres aléatoires pour positionner des objets de manière variée dans votre scène. Par exemple, pour positionner un objet à une position aléatoire dans une plage définie :

```
void Update()
{
    if(transform.position.y < -5.0f) // Si l'objet sort de l'écran en bas
    {
        float xAleatoire = Random.Range(-5.0f, 5.0f); // Position X aléatoire entre -5 et 5
        float yAleatoire = Random.Range(3.0f, 6.0f); // Position Y aléatoire entre 3 et 6 en haut
        transform.position = new Vector2(xAleatoire, yAleatoire);
        Debug.Log("Objet positionné à : " + transform.position);
    }
}
```

1.1.4 Générer des événements aléatoires

Vous pouvez également utiliser des nombres aléatoires pour déclencher des événements aléatoires dans votre jeu. Par exemple, pour décider aléatoirement si un ennemi doit apparaître ou non :

```
void Update()
{
    if (Random.Range(0, 100) < 10) // 10% de chance de générer un ennemi à chaque frame
    {
        Debug.Log("Un ennemi apparaît !");
        // Code pour générer un ennemi ici
    }
}
```

1.1.5 Choisir un élément aléatoire dans une liste

Vous pouvez également utiliser `Random.Range()` pour sélectionner un élément aléatoire dans un tableau ou une liste. Voici un exemple avec un tableau de chaînes de caractères :

```
public List<GameObject> ennemis; // Liste d'ennemis à assigner dans l'inspecteur

void GenererEnnemiAleatoire()
{
    int indexAleatoire = Random.Range(0, ennemis.Count); // Génère un index aléatoire
    GameObject ennemiChoisi = ennemis[indexAleatoire]; // Sélectionne l'ennemi à cet index
    Instantiate(ennemiChoisi, new Vector2(0, 0), Quaternion.identity); // Instancie l'ennemi choisi à la
    ↪ position (0,0)
    Debug.Log("Ennemi aléatoire généré : " + ennemiChoisi.name);
}

void Start()
{
    GenererEnnemiAleatoire(); // Appelle la fonction au démarrage
}
```

La classe Mathf

La classe `Mathf` dans Unity est une bibliothèque de fonctions mathématiques utiles pour les calculs courants dans le développement de jeux. Elle inclut des fonctions pour les opérations trigonométriques, les arrondis, les interpolations, et bien plus encore.

Voici quelques exemples d'utilisation courante de la classe `Mathf` :

- `Mathf.Abs(float value)` : Retourne la valeur absolue d'un nombre. Utilisé pour s'assurer qu'une valeur est positive comme lors du calcul de distances.
- `Mathf.Max(float a, float b)` : Retourne la valeur maximale entre deux nombres. Utile pour trouver la plus grande valeur entre deux variables.
- `Mathf.Min(float a, float b)` : Retourne la valeur minimale entre deux nombres. Utile pour trouver la plus petite valeur entre deux variables.
- `Mathf.Round(float value)` : Arrondit un nombre à l'entier le plus proche.
- `Mathf.Floor(float value)` : Arrondit un nombre à l'entier inférieur le plus proche.
- `Mathf.Clamp(float value, float min, float max)` : Contraint une valeur entre un minimum et un maximum. Utile pour limiter les valeurs dans une plage spécifique comme le personnage ne dépasse pas une certaine vitesse.
- `Mathf.Lerp(float a, float b, float t)` : Effectue une interpolation linéaire entre deux valeurs. Permet de trouver une valeur entre *a* et *b* en fonction de *t* ($0 \leq t \leq 1$). Sert pour des transitions douces.
- `Mathf.Sin(float f)` : Retourne le sinus d'un angle en radians. Utile lorsque vous travaillez avec des mouvements circulaires ou des oscillations.
- `Mathf.Cos(float f)` : Retourne le cosinus d'un angle en radians. Utile pour les mêmes raisons que `Mathf.Sin`.

2.1 Exemple d'utilisation de Mathf

```
float angle = 45.0f;
float radians = angle * Mathf.Deg2Rad; // Convertir en radians
float sineValue = Mathf.Sin(radians); // Calculer le sinus
Debug.Log("Le sinus de " + angle + " degrés est : " + sineValue);
transform.position = new Vector2(Mathf.Cos(radians), Mathf.Sin(radians)); // Positionner un objet en
    ↪ fonction du cosinus et du sinus

float vitesse = 15.0f;
vitesse = Mathf.Clamp(vitesse, 0.0f, 10.0f); // Contraindre la valeur entre 0 et 10
Debug.Log("Valeur contrainte : " + vitesse);

float tailleDepart = 0.0f;
float tailleFin = 100.0f;
```

```
transform.localScale = new Vector2(  
    Mathf.Lerp(tailleDepart, tailleFin, 0.5f), // Interpolation pour la largeur  
    Mathf.Lerp(tailleDepart, tailleFin, 0.5f) // Interpolation pour la hauteur  
); // Résultat : échelle à 50% entre 0 et 100, soit 50 Donne une transition douce entre deux tailles du  
    ↪ type ease-out  
Debug.Log("Échelle interpolée : " + transform.localScale);
```